

Acuerdos de Codificación y Nomenclatura en C++

ÍNDICE

Acuerdos de Codificación y Nomenclatura en C++.....	2
1. Convenciones de Nomenclatura (El Estándar Híbrido).....	2
1.1. Archivos del Sistema (snake_case).....	2
1.2. Clases y Estructuras (PascalCase).....	2
1.3. Funciones y Métodos (snake_case).....	3
1.4. Variables y Atributos de Clase.....	3
1.5. Constantes y Macros (UPPER_SNAKE_CASE).....	3
1.6. Instancias de Clase.....	4
1.7. Constructores y destructores.....	5
2. Principios Arquitectónicos Obligatorios.....	6

Acuerdos de Codificación y Nomenclatura en C++

Objetivo del Documento: Establecer un estándar de escritura de código unificado para el desarrollo de software en C++. Este documento no obedece a preferencias estéticas personales, sino a convenciones de la industria (alineadas con la *Google C++ Style Guide* y la Biblioteca Estándar de C++) y a principios de eficiencia algorítmica. Un código estandarizado es fundamental en el modelo GREENSOFT: reduce la deuda técnica, minimiza las refactorizaciones y evita ciclos de compilación inútiles por errores de portabilidad.

1. Convenciones de Nomenclatura (El Estándar Híbrido)

El ecosistema C++ no utiliza un único formato (*camelCase* o *snake_case*), sino un modelo híbrido donde cada convención tiene un significado semántico estricto.

1.1. Archivos del Sistema (*snake_case*)

Todos los archivos fuente (.cpp) y cabeceras (.hpp / .h) deben escribirse exclusivamente en minúsculas y separados por guiones bajos.

- **Correcto:** `http_server.hpp`, `procesador_datos.cpp`
- **Incorrecto:** `HttpServer.hpp`, `procesadorDatos.cpp`
- **Fundamento Técnico:** Los sistemas de archivos de la familia POSIX (como Ext4 en Linux) distinguen entre mayúsculas y minúsculas (*case-sensitive*), mientras que NTFS (Windows) no. Incluir un archivo como `#include "HttpServer.hpp"` cuando el archivo se llama `httpserver.hpp` compilará en Windows pero provocará un fallo crítico en los servidores de producción Linux. El uso estricto de *snake_case* garantiza una portabilidad absoluta.

1.2. Clases y Estructuras (*PascalCase*)

Los nombres de tipos definidos por el usuario (clases, structs, enums, typedefs) deben comenzar con mayúscula y usar formato de camello sin guiones bajos.

- **Correcto:** `class HttpServer`, `struct HttpRequest`
- **Incorrecto:** `class http_server`, `struct HTTPRequest`

- **Fundamento Técnico:** Permite distinguir visualmente e instantáneamente los tipos de datos creados por el desarrollador de los tipos de datos primitivos del lenguaje (int, char, float), los cuales siempre van en minúscula.

1.3. Funciones y Métodos (snake_case)

Toda función, método de clase o expresión lambda debe escribirse en minúsculas y separar las palabras con guiones bajos.

- **Correcto:** `parse_query_string()`, `listen_and_serve()`
- **Incorrecto:** `parseQueryString()`, `ListenAndServe()`
- **Fundamento Técnico:** Preserva la coherencia léxica con la Biblioteca Estándar de C++ (STL). Al interactuar con métodos nativos como `std::string::find()` o `std::vector::push_back()`, el uso de snake_case evita un código fragmentado donde conviven múltiples paradigmas visuales de forma caótica.

1.4. Variables y Atributos de Clase

Las variables locales y los parámetros de las funciones utilizan snake_case. Sin embargo, los atributos (variables miembro) privados de una clase deben llevar un sufijo de guion bajo (_).

- **Variables locales (Correcto):** `client_socket`, `bytes_read`
- **Atributos privados (Correcto):** `port_`, `routes_`, `console_mutex_`
- **Fundamento Técnico:** Elimina la ambigüedad en la capa de abstracción. El desarrollador sabe inmediatamente si está modificando el estado interno de un objeto (afectando la memoria de la instancia) o si solo está manipulando una variable temporal en el *stack*. Esto evita colisiones de nombres y reduce la necesidad de usar el puntero `this->` explícitamente.

1.5. Constantes y Macros (UPPER_SNAKE_CASE)

Los valores inmutables definidos mediante `constexpr`, `const` estáticos, o macros del preprocesador deben escribirse íntegramente en mayúsculas.

- **Correcto:** `const int MAX_BUFFER_SIZE = 2048;`
- **Incorrecto:** `const int maxBufferSize = 2048;`

1.6. Instancias de Clase

En C++, una instancia de clase (un objeto) no es más que una variable cuyo tipo ha sido definido por el usuario. Las convenciones de la industria no hacen distinción visual entre una variable de tipo primitivo (int, float) y un objeto complejo (HttpRequest, HttpServer). Por lo tanto, las instancias heredan exactamente las mismas reglas que definimos para las variables en el documento, variando únicamente según su alcance (*scope*) y mutabilidad:

El fundamento técnico es que, en C++, tratar a los objetos de forma visualmente distinta a las variables primitivas genera una carga cognitiva innecesaria al leer el código. Al ver `cliente_actual`, el desarrollador o revisor sabe inmediatamente que es una variable local en memoria transitoria, sin importar si internamente es un simple `int` (un descriptor de socket primitivo) o una instancia compleja de una clase `SocketRAII`. La convención describe **qué rol cumple la variable**, no qué tan compleja es su estructura interna.

Ejemplos:

Instancias como variables locales (**snake_case**):

// Correcto:

HttpServer servidor_local(8080);

HttpRequest peticion_entrante;

// Incorrecto:

HttpServer ServidorLocal(8080);

HttpRequest peticionEntrante;

Instancias como atributos de otra clase (**snake_case_**):

class GestorRed

{

private:

// Correcto: son instancias de clase, pero al ser atributos privados llevan el sufijo.

HttpServer servidor_principal_;

std::vector<HttpRequest> historial_peticiones_;

};

Instancias constantes o globales (**UPPER_SNAKE_CASE**):

```
const HttpResponse RESPUESTA_ERROR_DEFECTO;
```

1.7. Constructores y destructores

Por último es importante aclarar que los constructores y destructores utilizan PascalCase única y exclusivamente porque el compilador de C++ exige que sus identificadores sean una copia exacta del nombre de la clase. Esto no obedece a una regla de convención de estilo independiente, sino a una obligación estricta del lenguaje C++ (establecida en el estándar ISO/IEC 14882).

La regla dicta que el constructor debe tener exactamente el mismo identificador léxico que la clase, y el destructor igual, precedido por una tilde (~). Como nuestra guía de estilo establece que las clases se definen en PascalCase, los constructores y destructores heredan esta capitalización de forma colateral e ineludible.

```
class HttpServer
```

```
{
```

```
    public:
```

```
        // Correcto: Obligatoriamente en PascalCase porque la clase se llama HttpServer.
```

```
        // Quien exige esta coincidencia es el COMPILADOR, no el preprocesador.
```

```
        HttpServer(int port);
```

```
        ~HttpServer();
```

```
        // Incorrecto: El compilador lo evaluará como la declaración de un método regular
```

```
        // y arrojará error (ISO C++ forbid declaration of 'http_server' with no type).
```

```
        // http_server(int port);
```

```
};
```

2. Principios Arquitectónicos Obligatorios

1. **Gestión de Memoria (RAII - *Resource Acquisition Is Initialization*):** Queda estrictamente prohibido el uso de punteros desnudos (`new` / `delete`) o el manejo manual de descriptores de archivos sin encapsulación. Todo recurso de hardware (sockets, archivos, mutex) debe estar atado al ciclo de vida de un objeto. Si un objeto se destruye al salir de su ámbito, su destructor debe liberar el recurso automáticamente.
2. **Principio de Responsabilidad Única (SRP):** Un método, una función o un archivo debe resolver un solo problema de negocio. La lógica de red (sockets), la lógica de protocolo (HTTP) y la lógica de aplicación (rutas de la API) deben residir en capas distintas e incomunicadas directamente.
3. **Inmutabilidad por Defecto:** Si un parámetro o una variable no requiere ser modificada después de su inicialización, debe ser declarada como `const`. Si se pasa un objeto complejo a una función (como `HttpRequest`), debe pasarse como referencia constante (`const HttpRequest&`) para evitar la clonación innecesaria de datos en la memoria RAM, optimizando el consumo energético del hardware.